DataLoader Documentation

Felix Löhr

October 16, 2026

Abstract

DataLoader was originally intended to support loading data from a slew of complicated formats, enrich the data, and store it in a "data lake" in the database. New methods for loading were meant to be added as plugins.

Since then, the scope was shifted to broaden the understanding of core concepts via separate, distinct examples. The current version supports user management, dynamic entry manipulation, multi-level logging with database integration and filtering, and an interactive command-line interface with history tracking. Future iterations will focus on making the custom parser more generic, before (hopefully) returning to data ingestion.

The complete project, code and documentation can be found at:
https://www.dropbox.com/scl/fo/3h3w6ct9fo53y05wm9t9s/ACb-SiECpRQ9hr8xQTP9cxk?rlkey=g1xazaffam4m3d9mn56vgfssg&st=ehiyovu7&dl=0

Contents

1 Introduction & Project Evolution **2**

2 Software Architecture **2**
2.1 Architectural Layers 2
2.2 Modular Component Design 2
2.3 Dynamic Persistence & Data Access Strategy 4

3 Core Capabilities and Features **5**
3.1 User Management and Authentication 5
3.2 Dynamic Entry Manipulation 7
3.3 Multi-Window Logging System 7
3.4 Log Analytics and Database Integration 8
3.5 Interactive Command-Line Interface (CLI) 9

4 Future Outlook **10**

1 Introduction & Project Evolution

The **DataLoader** software was initially conceived as a robust data ingestion engine. Its primary objective was to parse highly complex data formats, enrich the incoming datasets, and store them securely within a central database system acting as a "data lake". To ensure scalability, the architecture was designed to support modular plugins for introducing new data loading methodologies.

However, following feedback and strategic alignment with project goals, the development focus shifted. Instead of building a monolithic ingestion pipeline, the software was repurposed to demonstrate and broaden the understanding of foundational software engineering and data management concepts. This was achieved by implementing diverse, separate, and distinct architectural examples within the application.

2 Software Architecture

This section provides a comprehensive overview of the architecture for **DataLoader**. It details the structural layering, the modular decomposition of tasks, and the dynamic persistence strategy designed to support flexible data management.

2.1 Architectural Layers

DataLoader is structured using a traditional three-tier architecture. This ensures a strict separation of concerns, decoupling the presentation layer from core business logic and underlying storage mechanisms. (See Figure 1).

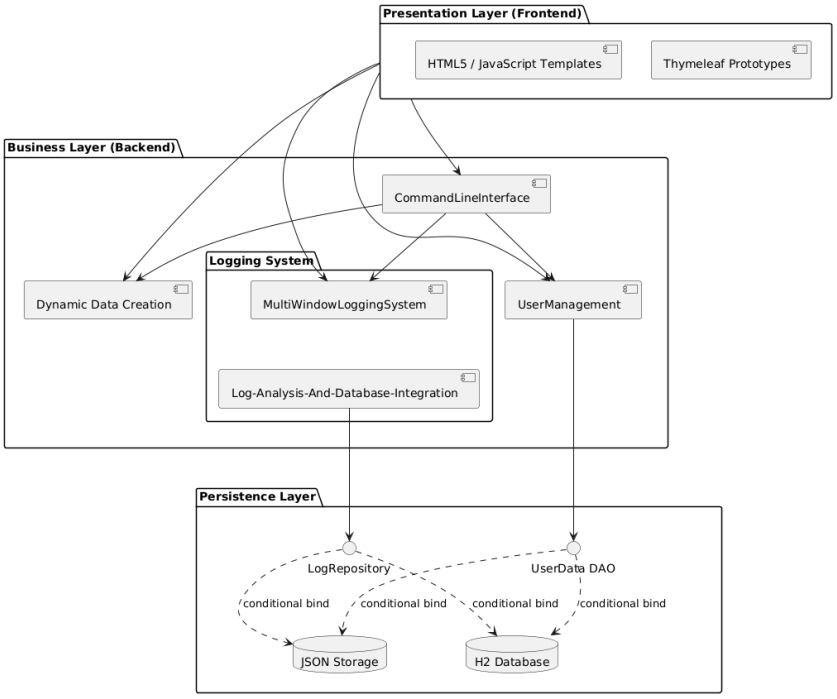
- **Presentation Layer (Frontend):** Built using standard HTML5 and JavaScript integrated with Thymeleaf prototypes. Thymeleaf allows the frontend templates to function both as static prototypes in a browser and as dynamically rendered views when processed by the backend. Some parts were easier to implement in JavaScript than in Thymeleaf.
- **Business Layer (Backend):** Serves as the central engine of the software. It handles the core computational logic, coordinates data orchestration, enforces business rules, and mediates traffic between the user interface and data storage.
- **Persistence Layer:** Responsible for data lifecycle management. It abstracts the storage engine, allowing DataLoader to read from and write to either flat JSON files or an embedded H2 relational database.

2.2 Modular Component Design

The backend architecture leans heavily into a modular design. The system is decomposed into distinct components isolated by specific business tasks to ensure high maintainability, testability, and scalability.

- **UserManagement:** This module isolates all routines connected to user identity and system authorization. It manages user registration, profile updates, and lifecycle operations. The userData managed here are used by the SpringBoot Security to enforce authentication and authorization, protecting application endpoints and verifying user permissions.

Figure 1: Frontend, Backend and persistence layer



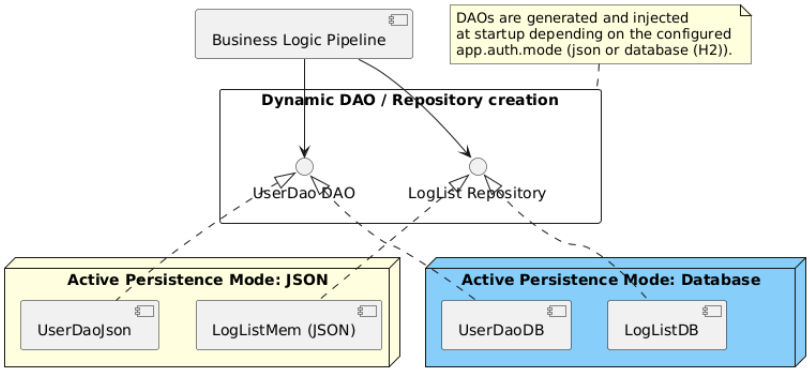
- **Logging System:** The multi-window logging system manages the creation, capturing, display, and persistence of logs and system telemetry across diverse application views. It enables simultaneous, real-time event tracking and will eventually accommodate a dedicated submodule for log and telemetry analysis.
- **CommandLineInterface (CLI):** An alternative way of using the application, without relying on a GUI. This is currently implemented as a simple command prompt, with history and copy, displayed in the browser. Eventually a submodule may support terminal environments as an alternative entry point to the application. This would allow users to script operations, trigger data generation or ingestion, and review logs without initializing the web-based graphical interface.
- **Configuration and Initialization:** Separate from their individual tasks, all configuration and initialization is combined into the config module.
- **Dynamic Data Creation:** This component houses the core utilities responsible for runtime data generation. It handles algorithms that construct mock, synthetic, or dynamic datasets based on user inputs or automated schedules before passing them to the business logic pipelines.

There are two noteworthy exceptions:

While standard features bundle controllers locally, the controllers module consolidates lightweight controllers that do not justify their own task modules. Specifically, `PageController` only serves static routes with zero execution logic, and `SimpleListController` handles the string list from subsection 3.2 directly – neither requires services, repositories, or models.

The other exception is a dedicated config module, which isolates all configuration and initialization from execution logic. This separates concerns, simplifies testing and maintenance, provides a single source of truth and prevents circular dependencies.

Figure 2: Dynamic selection of persistence layer



2.3 Dynamic Persistence & Data Access Strategy

DataLoader features a flexible persistence model that allows the system to seamlessly switch between persistence modes (currently JSON flat-files or an H2 Database) via configuration, without altering business logic. (See Figure 2).

To decouple the business layer from concrete storage engines, data interactions are funneled through DataAccessObjects (DAOs or repositories). During application initialization, the system evaluates the configured persistence mode. Based on this mode, the application generates and injects the corresponding implementation of the DAOs dynamically. The remaining software interacts exclusively with abstract interfaces, unaware of how the data is persisted or fetched.

This dynamic generation pattern applies strictly to the two primary data domains within the software:

1. **UserData DAO:** Abstractly manages all CRUD operations and queries for user profiles, credentials, and settings.
2. **LogRepository:** Abstractly manages all high-volume insertions, reads and caching for the event logs.

Neither the simple strings list, nor the command history are persisted. Therefore neither requires this dynamic layer.

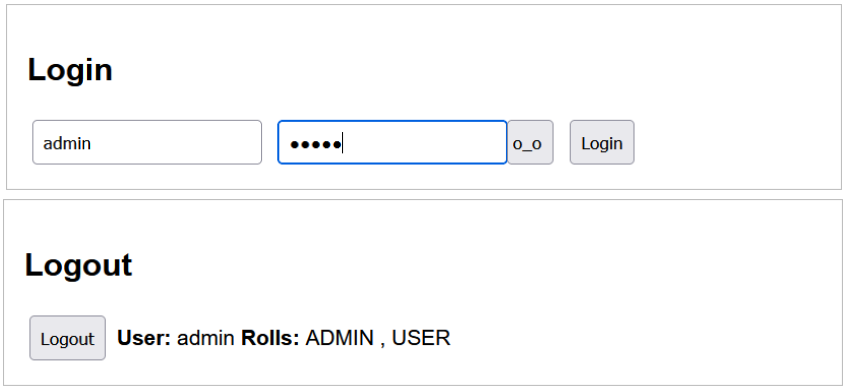
3 Core Capabilities and Features

The current implementation of DataLoader delivers five core functional pillars, detailed below.

3.1 User Management and Authentication

Every web service needs to handle authentication and security. In DataLoader passwords are handled by a `DelegatingPasswordEncoder` to support multiple types of password encoding. The default encoding for passwords is bcrypt.

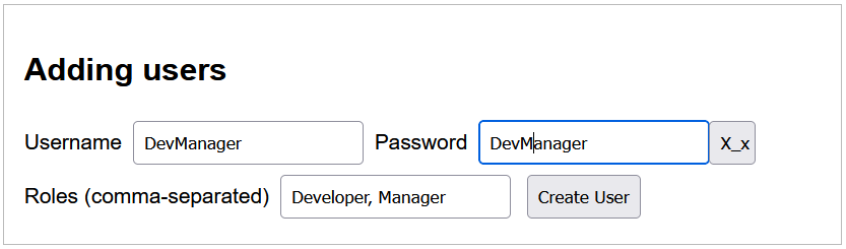
Figure 3: Panel for Login / Logout



Users can log in via the *Login* panel. Once logged in, the panel becomes the logout panel, displaying the logout button, the name and roles of the current user instead. (See Figure 3)

Please note that this software is designed exclusively for **single-user operation**. Concurrent sessions are not supported, meaning: *Only a single user can be logged in at any given time!*

Figure 4: Panel for user creation



Adding users can be done using the *Adding users* panel. User creation is restricted, by the backend, to administrators only. Further the panel hides its content from non-administrators.

After entering a username and password, one or more roles may be specified separated by commas (for example, `Developer, Manager`). The roles field is case-insensitive and ignores spurious spaces. Roles that dont exist will be ignored.

The password field accepts passwords either in clear text or already encoded in a way, which the password encoder understands (for example `{bcrypt}...` for bcrypt encoding). Clear text passwords are bcrypt-encoded before being stored in the database. (See Figure 4)

Please note that currently, eventhough hidden in the password field by default, the password will be *transferred in clear text!* This would need to be changed in any real application.

After creation, it is immediately possible to log in as the new user.

DataLoader provides a flexible user handling layer, capable of managing identities from multiple data providers. The `app.auth.mode` property in the `application.properties` file determines the active data provider. If omitted or set to `database`, the system defaults to database mode, in which user profiles are managed and persisted via a relational database. Setting it to `json` enables the lightweight JSON mode, which loads users from a JSON file.

Listing 1: Structured JSON format used for user ingestion.

```
[
  {
    "username": "user",
    "password": "{noop}user",
    "roles": ["USER"]
  },
  {
    "username": "admin",
    "password": "{noop}admin",
    "roles": ["USER", "ADMIN"]
  }
  {
    "username": "DevManager",
    "password": "{bcrypt}$2a$10$1NCnuqAK70Pk2BhzEtVHx.
     minjy1OHKYMBI06alXmOZ6hrHnO0T6u",
    "roles": 5
  }
]
```

In JSON mode, roles may also be passed as a single bitmask value in the JSON payload rather than an array of strings. Given the values for developer (1), admin (2), manager (4), poweruser (8), and user (16), the combined flag for [developer, manager] evaluates to 5. (See Listing 1)

Please note that the JSON mode is intended for testing and *does not support adding new users.*

Table 1: Database schema for user management

Table: `USERS`

Holds the core account credentials for each unique user.

Column	Data Type	Constraints
`id`	INT	Primary Key, Auto-Increment
`username`	VARCHAR(50)	Unique, Not Null
`password_hash`	VARCHAR(255)	Not Null

Table: `USER_ROLES`

Maps users to their assigned roles (one role per row per user).

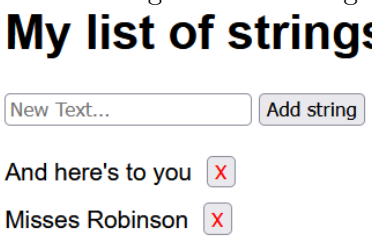
Column	Data Type	Constraints
`userid`	INT	Foreign Key (references `USERS.id`)
`role`	VARCHAR(30)	Not Null

In database mode, user data is split into two tables, `USERS` and `USER_ROLES`, with database handling managed transparently by the Jakarta Persistence API (JPA). (See Table 1)

3.2 Dynamic Entry Manipulation

DataLoader features a dynamic user interface component dedicated to string entry management. This task was implemented as an initial experiment with `Thymeleaf`, long before any other application logic – such as the user creation endpoint or user management even – was developed. Users can append new string items to the live list and remove them dynamically, using the x buttons. (See Figure 5)

Figure 5: Creating and deleting a list of strings



This was moved to a separate page, which only authenticated users may access. The data is volatile (only held in memory) and will be lost when restarting the software. The implementation uses `Thymeleaf` in a simple manner. As a result, it reloads the page to display the changes.

3.3 Multi-Window Logging System

This is a core feature of the originally intended software: The authorized user should be able to read and filter log messages, the logs had to update live, and logs were to be stored in the database so administrators could act on them later on. (See subsection 3.4)

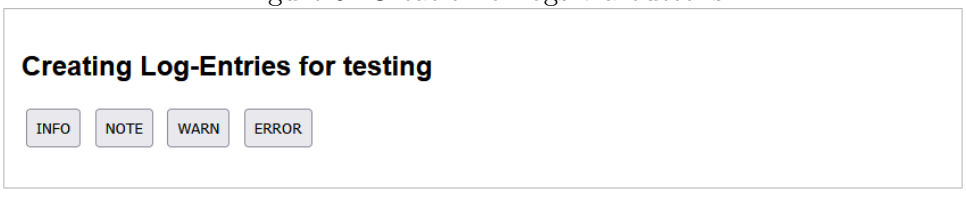
To facilitate robust debugging and auditing, the application decouples log generation from log display and from the main interface. The display has to act as a "live ticker" of logs, especially as it is intended to view actual logs from execution. This in turn raises its own requirements:

The page cannot be reloaded because of a log event. Otherwise any partial input would be lost... The system uses web-sockets to supply live updates to the logs display.

For testing, a way to create logs of multiple levels was necessary... A panel was set up to trigger a log entry for each currently supported loglevel at the click of a button. To test if the web-sockets really separated creation and update as intended, the same panel was duplicated into its own page to be opened from a different window. (See Figure 6)

- **Isolated Generation:** A separate application window dedicated entirely to triggering log entries provides a way of testing the log system and the live update capability.
- **Log Levels:** Full support for diverse severity levels (e.g., Info, Warning, Error) including custom levels ("Note").

Figure 6: Creation of logs via buttons



3.4 Log Analytics and Database Integration

All generated logs are captured and committed to the database. The logs are actual logs from the software, not just mock ups. The system includes a dedicated visualization panel allowing administrators to examine system health through:

- Comprehensive display grids.
- Advanced filtering criteria based on log levels, timestamps or log message contents.
- Multi-column sorting capabilities.

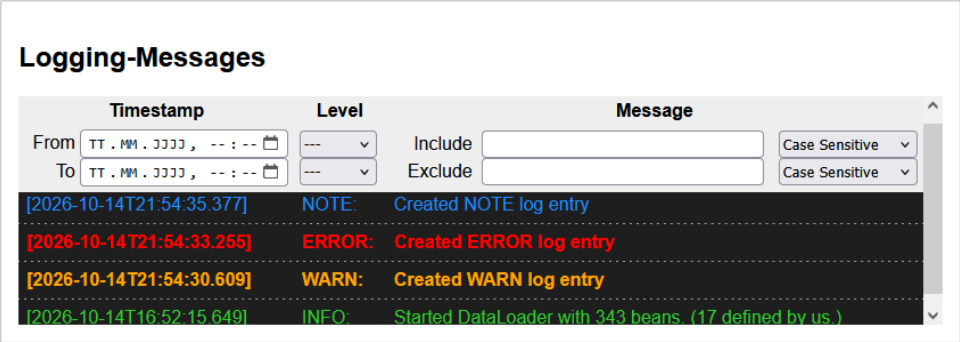
For performance a cache of the most recent 50 logs is kept in memory. This cache and the database access are handled by the `LogList` repositories. Depending on the `app.auth.mode`, either a `LogListDB` or a `LogListMem` exists and is used by the `RealLogAppender` component.

The component registers to the root logger and captures any log that was created by us. They are stored by the LogList repository and broadcast to the web socket for the browser to pick them up and update the view. This broadcast is all that is necessary for the *live* update.

However one may need to refresh the page when logs may already exist. For this a list request is sent through the `LogService`, forwarded through the component and fullfilled by the LogList repository. This, as well as internal use (e.g. by the command line in subsection 3.5) is where the cache can make sense. The database access itself is handled via springboot JDBC/JPA.

Side note: Creation of logs through the GUI is handled by the service logging directly. Since the RealLogAppender is registered to the root logger, it gets its updateds from there.

Figure 7: Display and filtering of logs



The display must be scrollable and easy to search/filter through. This implies filtering based on log level, log message and time stamp. Also it should be most-recent-first or even most recent on separate line to keep the user up-to-date. This is implemented as a scrollable table, which can be seen in Figure 7. The filtering otions are put right in their respective columns header.

Currently filtering is achieved mostly via JavaScript (on client side), which implies performance hits. Future implementations would pass most of the filtering into the backend, where parts of it may be done by the database call itself. However this will complicate caching as well.

The display panel is currently only available to Administrators. This is a remnant from testing the user authentication system. In future implementations, it would be sensible to let the common user see *some, but not all* log entries. Furthermore, being able to remove/clear *some, but not all* logs may be prudent; and convenient, if made possible from the panel.

3.5 Interactive Command-Line Interface (CLI)

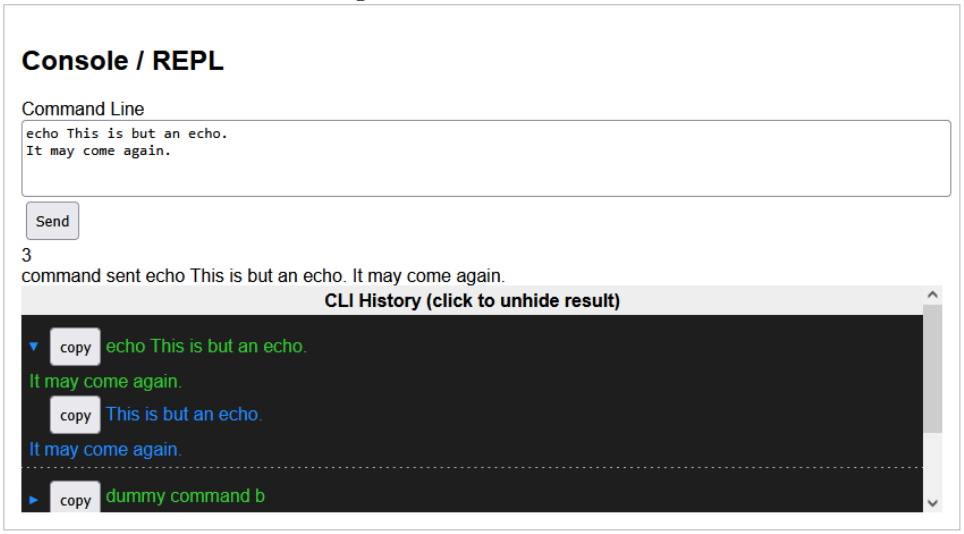
For advanced operations, an integrated command line provides a powerful interactive environment. Key features include:

- A persistent command history tracker displaying older executions.
- Side-by-side visualization of past commands alongside their direct outputs.
- Commands and results from the history can be copied by a button click.
- Each command can be (and by default is) collapsed to a single line.
  Doing so hides their result as well.
- The command is colored red if it failed, green if it succeeded and orange if there was some other response (failed but no error, e.g. a known but unimplemented call).

Available commands include adding users and creating logs, as well as help on the system entirely and help on the individual commands. Also "echo" to echo as usual on a linux shell.

Partially implemented but not available yet are SQL calls, modification and removal of users and removal of logs. These must be hardened against abuse, or restricted to administrator use only. Currently the entire CLI is only available to administrators.

Figure 8: Command line



A sensible use might be the command `log -list -all` to get all logs or `log -list INFO` to list all recent logs of log level INFO.

4 Future Outlook

The next major evolutionary phase for DataLoader involves upgrading the ingestion layer. The current custom parsing subsystem for the data ingestion layer will be refactored to be significantly more generic, allowing it to handle arbitrary, highly varied schemas without requiring much specialized plugin logic for every new format. Simply declaring a schema and specialized data extraction/enrichment rules should suffice.

The current custom parser for the CLI will be refactored to be significantly more generic. This will allow adding new commands as plugins, and simplify/unify the way of parsing.

Features that would improve the usability would be the pinning of logs, the filtering of the CLI history, and the option to delete either.